Date	
Team ID	
Project Name	Smart Metro Ticket Booking System
Maximum Marks	4 Marks

SMART METRO TICKET GENERATING SYSTEM

Solution Architecture

1. Architectural Overview

The Smart Metro Ticket Generating System is built as a layered, cloud-native architecture that separates concerns across five tiers: Client, Gateway, Service, Data, and External Integration. This separation allows each tier to scale, deploy, and fail independently — a critical property for a system that must remain available across hundreds of stations and tens of thousands of concurrent commuters during peak hours.

The architecture follows a microservices pattern: each core capability (authentication, ticketing, payment, reporting) is implemented as an independently deployable service communicating over well-defined APIs and asynchronous events. This directly supports the project's hardest constraints — sub-second gate response under peak load, and the ability to add new payment or fare-policy capabilities without redeploying the entire platform.

1.1 Layered Architecture

The diagram below shows the four logical layers of the solution — Presentation, Application/API, Data, and Infrastructure — and the concrete technologies used to implement each. This is the primary architectural reference used across the technology stack, requirements, and deployment documentation.

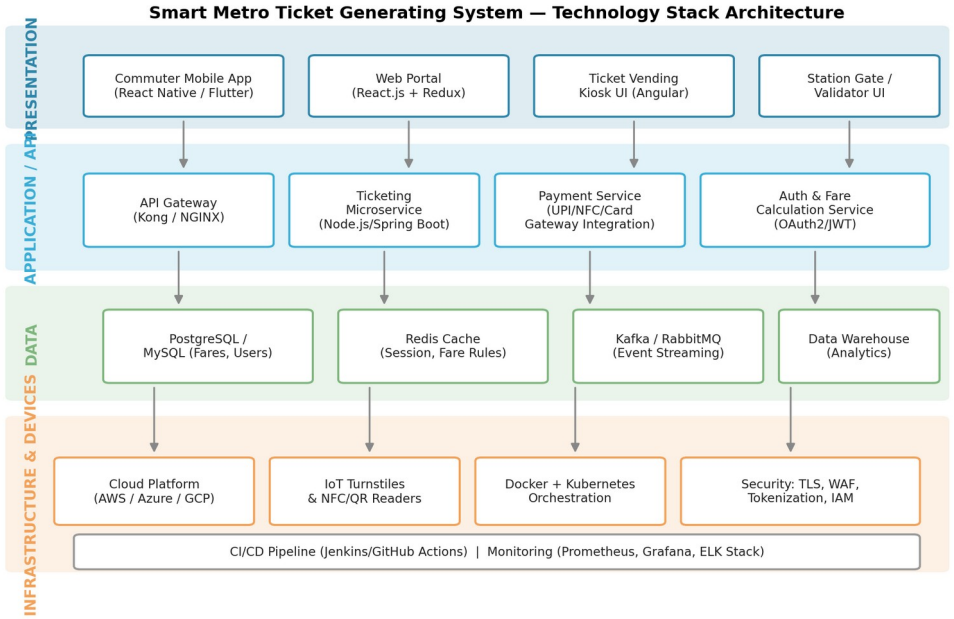

Figure 1: Smart Metro Ticket Generating System — layered solution architecture

1.2 Design Rationale

- Cloud-native from day one: every service is containerized and deployed on managed Kubernetes, avoiding lock-in to any single data center and enabling elastic scaling during festival or event-driven ridership spikes.
- API-first: all internal and external interactions happen through versioned REST APIs behind the gateway, so client apps, kiosks and future partner integrations share one contract.
- Polyglot persistence: a relational store handles transactional integrity, a cache absorbs read-heavy fare lookups, and an event log decouples services — each data technology chosen for the access pattern it serves best, not a one-size-fits-all database.

2. System Context & Component Deployment

The context diagram below fixes the system boundary, identifying the four external actors the architecture must serve — Commuter, Payment Gateway, Station Staff and Metro Administrator — and the data exchanged with each.

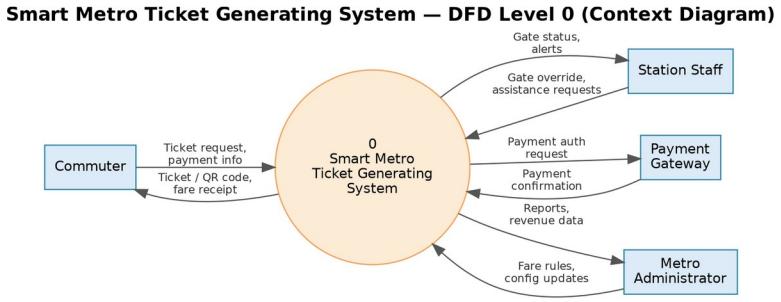

Figure 2: System context — external actors and data exchanged with the system

At deployment time, each logical layer maps onto concrete infrastructure. Client applications reach the system through a single Load Balancer / API Gateway entry point, which fronts a Kubernetes cluster running the core microservices as independently scalable pods. Data services run in a dedicated data tier with replication, and the system integrates outward to external payment and notification providers.

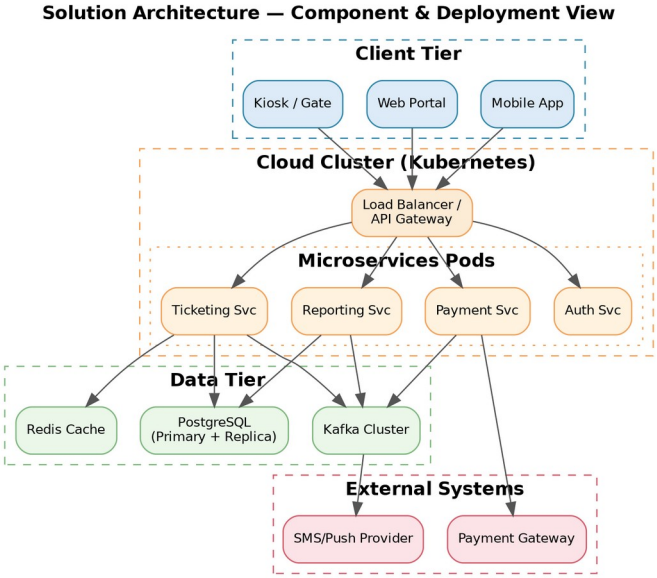

Figure 3: Component and deployment diagram showing tiers, pods, data stores and external integrations

2.1 Architectural Tiers

Tier	Components	Responsibility
Client Tier	Mobile App, Web Portal, Kiosk, Gate UI	User interaction, ticket display, offline QR cache
Gateway Tier	API Gateway / Load Balancer	Routing, TLS termination, rate limiting, auth check
Service Tier	Auth, Ticketing, Payment, Reporting microservices	Business logic, independently deployable pods
Data Tier	PostgreSQL, Redis, Kafka	Persistence, caching, event streaming
External Tier	Payment Gateway, SMS/Push Provider	Third-party integrations outside system boundary

3. Scalability, Principles and Integration

3.1 Scalability and High Availability

Every tier tolerates the loss of an individual instance without commuter-visible downtime. Stateless services scale horizontally behind the API Gateway; stateful services rely on managed replication. The flow below shows how the platform reacts automatically to load spikes and component failures.

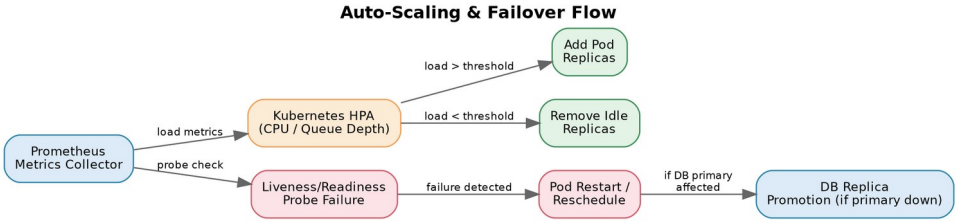

Figure 4: Auto-scaling and failover flow across the cluster and data tier

3.2 Architectural Principles

The following principles guided every architectural decision in the Smart Metro Ticket Generating System, and map directly to the non-functional requirements defined for the project:

- Scalability by decomposition: each microservice scales independently via Kubernetes Horizontal Pod Autoscaling based on CPU and request-queue depth, so a payment surge does not starve the reporting service of resources.
- Resilience: the database runs primary/replica replication; Kafka provides at-least-once delivery so a temporary consumer outage never loses a ticket-issued or payment event.
- Security in depth: a WAF and TLS termination sit at the gateway, JWT-based auth scopes every request, and payment data is tokenized at capture so no single compromised component exposes the full transaction chain.
- Loose coupling: services communicate synchronously through the API Gateway for commuter-facing calls, and asynchronously through Kafka for side effects (notifications, analytics), minimizing cascading failures.
- Auditability: every gate event, fare-rule change and administrative action is written to an immutable log for compliance and dispute resolution.

3.3 Integration Patterns and Summary

The architecture uses two integration patterns: synchronous REST/HTTPS calls through the API Gateway for anything the commuter is actively waiting on (fare lookup, payment authorization, gate validation), and asynchronous event publication through Kafka for anything that can be processed after the fact (notifications, analytics, fraud-pattern detection). This split keeps the commuter-facing path fast while still capturing every event needed for downstream reporting.

Together, the layered logical view, the system context, and the component/deployment view give both business and engineering stakeholders a shared reference for how the Smart Metro Ticket Generating System is built, deployed, secured and scaled in production.